

HARNESSBANDIT: JOINT LEARNABILITY–TRANSFERABILITY SCHEDULING FOR MULTI-HARNESS AGENTIC REINFORCEMENT LEARNING

Hongliang Wei^{1,2,*} Xiaobing Tu^{2,*,†} Yinggui Wang² Zhengxi Liu²
Rongkun Xue² Jinkui Ren² Xiantao Zhang² Debin Zhao¹ Xiaopeng Fan^{1,†}

¹ Harbin Institute of Technology

² Alibaba Cloud

* Equal contribution. † Corresponding authors.

ABSTRACT

Language-model agents are increasingly deployed through diverse *harnesses* that differ in system prompts, tool schemas, control loops, and trajectory formats. The same model can perform unevenly across these interfaces, making robustness to harness variation an important objective. A natural approach is to train a shared policy through multiple harnesses, but doing so introduces a scheduling problem: each training step should favor a harness that currently provides a useful learning signal while also producing an update that benefits the other harnesses. We develop **HarnessBandit**, an online scheduler that selects one harness per optimizer step. After a group-relative policy optimization (GRPO) update, it observes *learnability*—the mean absolute advantage on the batch—and *transferability*—the cosine between a low-dimensional gradient sketch of the current harness and exponential moving averages of the remaining harnesses. The two signals are fused after pooled sliding-window min–max normalization and sampled with a visit-dependent bonus and an explicit exploration floor. We train Qwen3.5–2B across six harnesses on ClawGym and evaluate on PinchBench (held-out tasks, in-distribution OpenClaw) and ClawEval (held-out tasks and harness). HarnessBandit improves over mixed-batch multi-harness training on both benchmarks, while training diagnostics indicate that learnability and transferability provide distinct, evolving signals.

1 INTRODUCTION

Language-model agents interact with tools through a *harness*: a combination of system prompt, tool schema, planner or loop policy, and session bookkeeping that turns a user query into a trajectory of completions and tool calls (Yao et al., 2023; Yang et al., 2024; Wang et al., 2025). Different harnesses induce different trajectory distributions even when the underlying task and reward function are held fixed. As a result, a policy trained exclusively through one interface need not transfer to another. Training with multiple harnesses can reduce this sensitivity, but requires deciding how their data should be allocated within and across optimizer steps.

This decision is not a standard multi-task mixture problem. The same ClawGym task can be executed under OpenClaw, Codex, Gemini CLI, or another scaffold; the *task* identity is shared, while the *execution channel* changes. We consider two properties of that channel. First, a batch is useful only if it produces a non-degenerate learning signal. Under group-relative policy optimization (Shao et al., 2024; DeepSeek-AI, 2025), a prompt group whose sampled trajectories receive identical rewards yields advantages near zero and therefore almost no policy gradient. We call this *learnability*. Second, an update computed on one harness may align with directions that other harnesses also require, or it may overfit one scaffold. We call the former *cross-harness transferability*. A static allocation of training steps responds to neither quantity.

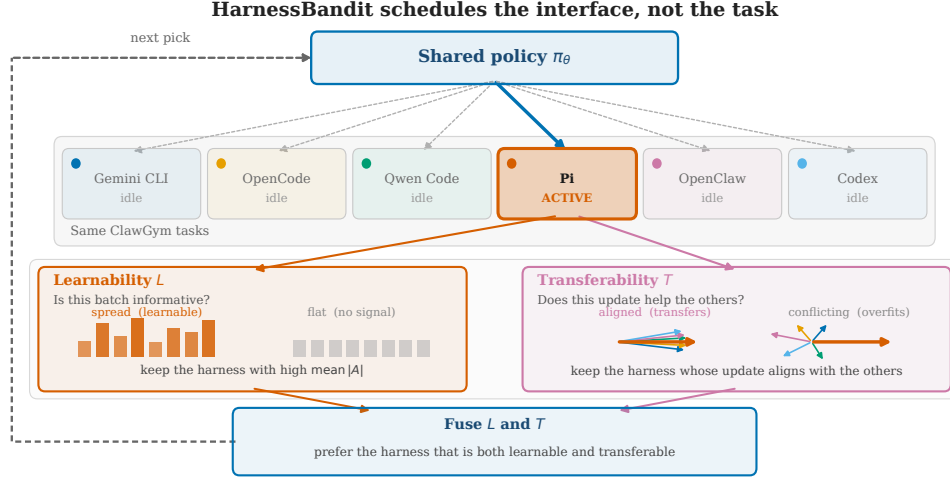


Figure 1: Principle of HarnessBandit. A shared policy sees the same tasks through six harnesses; only one harness is active per optimizer step. Learnability is the within-group reward spread (a usable mean $|A|$). Transferability is whether the current update aligns with sketches of the idle harnesses. The two scores are fused to choose the next interface. Glyphs are schematic.

We formulate multi-harness training as a non-stationary multi-armed bandit problem, where each harness is an arm and its online utility jointly reflects learnability and cross-harness transferability. We introduce **HarnessBandit**, a scheduling layer on top of GRPO rather than a new policy objective. At each step it selects one harness for a pure rollout batch, measures how much learning signal the batch provides and how well its update aligns with the other harnesses, and combines these observations to adapt the subsequent sampling distribution while retaining exploration.

We train a *Qwen3.5-2B* policy (Qwen Team, 2026) with Dr.GRPO (Liu et al., 2025) on 2,000 ClawGym tasks (RUC-AIBOX, 2025) crossed with six harnesses (12,000 prompt rows), and evaluate on PinchBench (held-out tasks under OpenClaw, an in-distribution harness) and ClawEval (held-out tasks under ClawEval’s native loop, an out-of-distribution harness). HarnessBandit obtains a PinchBench mean of 0.506, compared with 0.493 for a two-harness OpenClaw–Codex mixture and 0.331 for a six-harness mixture (untuned base 0.330). On ClawEval (161 tasks, $N=3$), it reaches 41.6% pass@3 and mean score 0.556, compared with 38.5% / 0.510 and 29.8% / 0.490; Pass³ is 26.7%, matching the base model and above both mixtures.

Contributions.

1. We formulate multi-harness training as a non-stationary bandit over *execution interfaces*, rather than as a fixed mixture over tasks or domains.
2. We propose two complementary trainer-native signals—local learnability and cross-harness transferability—and combine them so the scheduler prefers updates that are both informative and aligned with other interfaces.
3. We conduct experiments on PinchBench (held-out tasks, in-distribution OpenClaw) and ClawEval (held-out tasks and harness), where HarnessBandit improves over static harness mixtures under a matched GRPO budget without collapsing to a single arm.

2 RELATED WORK

Agentic reinforcement learning and tool use. Tool-using language agents interleave reasoning with environment actions (Yao et al., 2023; Schick et al., 2023; Qin et al., 2024). Subsequent work has treated the *agent–computer interface* itself as a first-class design object: SWE-agent showed that scaffolding choices change success rates on software-engineering tasks (Yang et al., 2024), OpenHands and SWE-Gym provide open stacks for training and evaluating such agents (Wang et al.,

2025; Pan et al., 2024), and web and personal-assistant benchmarks stress long-horizon tool use (Zhou et al., 2024; Yao et al., 2024; Liu et al., 2024). Reinforcement learning has been applied to reasoning traces (Ouyang et al., 2022; Shao et al., 2024; DeepSeek-AI, 2025; Yu et al., 2025) and, more recently, to agent trajectories with programmatic rewards. Polar (Xu et al., 2026) exposes multiple harnesses as interchangeable rollout backends behind a shared trainer. We use that interface. Our question is orthogonal to the choice of policy gradient: given several harnesses, which one should generate the next on-policy batch.

Multi-task learning and conflicting updates. When a shared model is trained on several sources, gradients can interfere (Caruana, 1997; Ruder, 2017; Sener & Koltun, 2018; Yu et al., 2020). Multi-task RL methods distill shared skills or regularize policies toward a common centroid (Parisotto et al., 2016; Teh et al., 2017; Barreto et al., 2017). Those approaches typically assume a fixed sampling distribution over tasks. In our setting the “tasks” are harnesses that wrap the *same* underlying ClawGym instances; interference is induced by interface mismatch rather than by disjoint objectives. HarnessBandit does not project or surgically alter gradients. It chooses which harness to roll out so that the observed update is both informative and aligned with other interfaces.

Curriculum learning and data mixing. Curriculum learning sequences training material by difficulty or progress (Bengio et al., 2009; Graves et al., 2017; Narvekar et al., 2020; Portelas et al., 2020). In language-model pretraining, mixture weights over domains are often tuned offline or by a proxy model (Xie et al., 2023; Albalak et al., 2024). Most such methods do not observe whether an on-policy source currently yields a non-zero advantage or whether its gradient aligns with other sources. A related two-signal bandit has been used to schedule reasoning domains (Yang et al., 2026). We instead allocate training steps across execution harnesses that wrap the same task collection.

Bandit allocation. Classical bandits allocate pulls under uncertainty (Auer et al., 2002; Lattimore & Szepesvári, 2020). We use an exponentially weighted value, a visit-dependent bonus, Boltzmann sampling, and a uniform floor; $c/\sqrt{n_h + 1}$ is an implementation choice rather than a new regret result. The contribution is the execution-harness instantiation and its systems integration.

Gradient similarity and sketches. Cosine similarity and related kernel alignments are standard probes of whether two updates or representations point in the same direction (Yosinski et al., 2014; Kornblith et al., 2019). Storing one full gradient per harness is impractical at even 2B scale. Linear sketches, including coordinate subsampling and Rademacher projections, preserve inner products in the sense of the Johnson–Lindenstrauss lemma (Johnson & Lindenstrauss, 1984; Achlioptas, 2003; Charikar et al., 2004). HarnessBandit uses a fixed, layer-proportional subsample of the last four transformer blocks so that cosine comparisons remain well-defined across steps without materializing 190M-dimensional buffers.

3 PROBLEM FORMULATION

Tasks and harnesses. Let $\mathcal{X}$ be a set of agent tasks. Each task $x \in \mathcal{X}$ specifies a user query, an initial workspace, and a scalar reward $r \in [0, 1]$ computed by a programmatic checker after the session ends. A *harness* $h \in \mathcal{H}$ is an execution interface: it wraps x with a system prompt, a tool schema, and a control loop, and it induces a trajectory distribution $\tau \sim \pi_\theta(\cdot \mid x, h)$ for a policy π_θ . Two harnesses applied to the same x can produce different tokenizations, different tool-call syntax, and different reward realizations.

Training uses a finite product $\mathcal{D}_{\text{train}} = \{(x, h) : x \in \mathcal{X}_{\text{train}}, h \in \mathcal{H}_{\text{train}}\}$. Evaluation uses disjoint task collections $\mathcal{X}_{\text{eval}}$ (PinchBench and ClawEval) that are never seen during optimization. PinchBench is executed under OpenClaw, which belongs to $\mathcal{H}_{\text{train}}$; ClawEval uses a native evaluation loop that does not. The intended generalization is therefore across held-out tasks, and, for ClawEval, across interfaces as well.

On-policy update. We train with group-relative policy optimization (Shao et al., 2024). For a prompt (x, h) the rollout stack draws G trajectories, scores them, and forms advantages $A_{i,t}$ by subtracting a leave-one-out group baseline. Following Dr.GRPO (Liu et al., 2025) we do not divide by the group standard deviation, which avoids exploding updates when intra-group reward variance

is near zero. The policy loss at step t is the usual clipped surrogate plus a KL penalty toward a frozen reference,

$$\mathcal{L}_t(\theta) = \mathbb{E}[-\min(\rho_{i,t} A_{i,t}, \text{clip}(\rho_{i,t}, 1 - \epsilon, 1 + \epsilon) A_{i,t})] + \beta_{\text{KL}} \text{KL}(\pi_\theta \parallel \pi_{\text{ref}}), \quad (1)$$

where $\rho_{i,t}$ is the importance ratio. Nothing in HarnessBandit depends on this particular surrogate; any on-policy method that yields per-token advantages and a batch gradient can supply the two observations below.

Scheduling objective. Under the bandit abstraction, the arms are $\mathcal{H}_{\text{train}}$ and one arm is selected at each optimizer step. Because the shared policy changes after every update, arm utilities evolve with θ_t rather than following fixed reward distributions. A scheduler is a (possibly randomized) rule σ that, given history $\mathcal{F}_{t-1}$, selects $h_t \in \mathcal{H}_{\text{train}}$. The rollout then uses only h_t for every prompt in the batch. We want σ to trade off two instantaneous properties of h :

$$L_h(\theta) = \mathbb{E}[|A| \mid h, \theta], \quad (2)$$

$$T_h(\theta) = \mathbb{E}\left[\frac{1}{|\mathcal{H}_{\text{train}} \setminus \{h\}|} \sum_{j \neq h} \cos(\nabla_\theta \mathcal{L}(h), \bar{g}_j) \mid h, \theta\right], \quad (3)$$

where $\bar{g}_j$ is a running estimate of the gradient direction associated with harness j . L_h is large when the group contains both successful and unsuccessful trajectories, so the policy gradient is informative. T_h is large when an update on h is aligned with the directions previously observed on other harnesses. A scheduler that maximizes only L_h can concentrate on a harness whose advantages are large but idiosyncratic. A scheduler that maximizes only T_h can prefer a harness that is well aligned yet currently uninformative ($L_h \approx 0$). HarnessBandit estimates both quantities online and selects h_t to increase a convex combination of their normalized values, subject to exploration.

The quantity we ultimately care about is not L_h or T_h themselves, but expected reward on $\mathcal{X}_{\text{eval}}$. The two signals are instruments: they are cheap to compute from the training step and are intended to allocate a fixed step budget toward updates that are both usable and shareable.

4 METHOD

HarnessBandit wraps an unchanged GRPO trainer with an online harness scheduler. Figure 1 states the principle: the same tasks are executed through several interfaces, only one interface is active at a step, and the next draw depends on both local learnability and cross-harness transferability. Algorithm 1 gives the per-step procedure. After each update the scheduler records a local learnability signal and a sketched-gradient transferability signal, fuses them after pooled sliding-window min-max normalization, and draws the next harness. The implementation uses a proportional coordinate subsample, per-selected-step signal updates, $\tau=0.3$, and an explicit ε -floor.

4.1 HARNESS-PURE BATCHES

Let $\mathcal{H}_{\text{train}} = \{h^{(1)}, \dots, h^{(H)}\}$. Each harness maintains an independent shuffled pool of the same ClawGym tasks, so harness identity is not confounded with task difficulty. At step t the scheduler returns one name h_t ; every prompt in the rollout batch of size B is drawn from the pool of h_t . Purity has two consequences. First, the batch gradient is attributable to a single arm, so a cosine against other arms is well-defined. Second, leftover groups produced by oversampling filters can be stashed per harness rather than discarded, which keeps the comparison with a mixed-batch baseline honest on the number of scored trajectories.

4.2 LEARNABILITY

After advantages are computed, each trajectory is reduced to a token-masked mean of $|A_{i,t}|$. Trajectory means that share a prompt group are averaged, and group means that share a harness are averaged:

$$L_{h_t} = \frac{1}{|\mathcal{G}_{h_t}|} \sum_{g \in \mathcal{G}_{h_t}} \frac{1}{|\mathcal{T}_g|} \sum_{\tau \in \mathcal{T}_g} \overline{|A|}_\tau. \quad (4)$$

Equal weighting at the group level prevents long trajectories from dominating the signal. If every trajectory in every group of h_t receives the same reward, then $L_{h_t} \approx 0$ and the step is treated as uninformative even if the absolute reward is high.

4.3 TRANSFERABILITY VIA GRADIENT SKETCHES

Let $g_h = \nabla_{\theta} \mathcal{L}$ be the batch gradient after all microbatches of a pure h -step have been backwarded and before the optimizer consumes them. We retain only parameters in the last $K=4$ transformer blocks. Embeddings and the output head are excluded: their gradients are dominated by token frequency and therefore track which tasks were sampled rather than how the harness pushes the policy. The selected tensors still contain on the order of 10^8 coordinates. A sketch $\hat{g}_h \in \mathbb{R}^d$ with $d=65,536$ is formed by a fixed random subsample whose per-tensor allocation is proportional to $\text{numel}(\cdot)$. Proportional allocation keeps each tensor’s contribution to $\|\hat{g}_h\|_2^2$ scaled by the same constant, so the sketched cosine is not biased by layer shape. Across data-parallel and context-parallel ranks the sketch is summed and renormalized; cosine similarity is scale-invariant, so the subsequent division by world size is only for interpretability.

Each harness stores an exponential moving average EMA_j of its unit sketch on CPU. After the current sketch is reduced,

$$T_{h_t} = \frac{1}{|J|} \sum_{j \in J} \cos(\hat{g}_{h_t}, \text{EMA}_j), \quad J = \{j \in \mathcal{H}_{\text{train}} : j \neq h_t, \text{EMA}_j \text{ exists}\}. \quad (5)$$

If J is empty—as it is on the first visit to each arm— T_{h_t} is left undefined and the score below falls back to learnability alone. The EMA is then updated with rate $\eta=0.3$.

4.4 NORMALIZATION, FUSION, AND VALUE UPDATE

L_h and T_h have different units. Each is min–max normalized over a sliding window of the last $W=32$ raw observations, *pooled across harnesses*:

$$\tilde{L}_h = \text{clip}\left(\frac{L_h - \min \mathcal{W}_L}{\max \mathcal{W}_L - \min \mathcal{W}_L}, 0, 1\right), \quad (6)$$

and likewise for $\tilde{T}_h$. Per-arm windows would erase the between-arm contrasts that the bandit needs. If a window contains fewer than two values, the normalized coordinate is set to $1/2$. The instantaneous score is

$$S_h = \begin{cases} \alpha \tilde{L}_h + (1 - \alpha) \tilde{T}_h & \text{if both are defined,} \\ \tilde{L}_h \text{ or } \tilde{T}_h & \text{if only one is defined.} \end{cases} \quad (7)$$

We use $\alpha=0.5$. The per-arm value is an EMA, $Q_h \leftarrow (1 - \beta)Q_h + \beta S_h$ with $\beta=0.3$. An unvisited arm is assigned $\max_j Q_j$ at selection time so that softmax cannot starve it before the first observation.

4.5 SAMPLING

The quantity that is ranked is $U_h = Q_h + c/\sqrt{n_h + 1}$ with $c=0.2$. The sampling distribution is

$$p_h = (1 - \varepsilon) \text{softmax}(U/\tau)_h + \varepsilon/H, \quad \tau=0.3, \varepsilon=0.15. \quad (8)$$

The temperature τ controls how sharply the curriculum concentrates; the floor ε guarantees a minimum rate that does not depend on n_h . The bonus $c/\sqrt{n_h + 1}$ is the term that actually prefers under-sampled arms. Because the bonus does not grow with t (unlike UCB1’s $\sqrt{\ln t/n}$), an arm that stops being drawn keeps a frozen bonus and cannot rescue itself. That is why ε and c are not interchangeable.

The first $W_{\text{up}}=30$ steps ignore p and cycle through $\mathcal{H}_{\text{train}}$ in round-robin order, so every arm has at least five observations before the bandit takes over.

4.6 COMPLEXITY

Relative to an existing GRPO step, the extra work is (i) a gather of d gradient coordinates on K layers, (ii) an all-reduce of a d -vector, and (iii) $O(Hd)$ cosine evaluations. With $d=65,536$ and $H=6$,

Algorithm 1 One HarnessBandit training step

Require: t, θ_t , pools $\{\mathcal{D}_h\}_{h=1}^H$, values Q , visits n , sketch EMAs E

```
1: if  $t < W_{\text{up}}$  then
2:    $h_t \leftarrow h^{(t \bmod H)}$  ▷ round-robin warmup
3: else
4:    $U_h \leftarrow Q_h + c/\sqrt{n_h + 1}$ ;  $p_t(h) \leftarrow (1 - \varepsilon) \text{softmax}(U/\tau)_h + \varepsilon/H$ 
5:    $h_t \sim p_t$ 
6: end if
7:  $\mathcal{B}_t \leftarrow \text{ROLLOUT}(\mathcal{D}_{h_t}, \pi_{\theta_t}, B, G)$ 
8: compute GRPO advantages  $A_t$ , loss  $\mathcal{L}_t$ , and  $L_{h_t} \leftarrow \text{GROUPMEANABS}(A_t)$ 
9:  $\hat{g}_{h_t} \leftarrow \text{SKETCHANDREDUCE}(\nabla_{\theta} \mathcal{L}_t, K, d)$ 
10:  $J \leftarrow \{j \neq h_t : E_j \text{ is available}\}$ 
11: if  $J \neq \emptyset$  then
12:    $T_{h_t} \leftarrow |J|^{-1} \sum_{j \in J} \cos(\hat{g}_{h_t}, E_j)$ 
13: end if
14: normalize available  $L_{h_t}, T_{h_t}$ ; form  $S_{h_t}$  by Eq. 7
15:  $Q_{h_t} \leftarrow (1 - \beta)Q_{h_t} + \beta S_{h_t}$ ;  $n_{h_t} \leftarrow n_{h_t} + 1$ 
16:  $E_{h_t} \leftarrow \text{UNITEMA}(E_{h_t}, \hat{g}_{h_t})$ 
17:  $\theta_{t+1} \leftarrow \text{OPTIMIZERSTEP}(\theta_t, \nabla_{\theta} \mathcal{L}_t)$ 
```

the EMA store occupies a few megabytes rather than several gigabytes. The policy loss, rollout, and reward evaluation are unchanged. HarnessBandit is therefore a constant-factor add-on to an existing GRPO run, not a second trainer.

5 EXPERIMENTS

5.1 TRAINING SETUP

Data. Training uses 2,000 ClawGym tasks (RUC-AIBOX, 2025). Each task directory contains a user query, optional workspace files, and a `reward.sh` checker that prints a scalar in $[0, 1]$. We form a Cartesian product with six training harnesses—Gemini CLI, OpenCode, Qwen Code, Pi, OpenClaw, and Codex—yielding 12,000 prompt rows. Every method therefore sees the same task universe; harness identity is not confounded with difficulty.

Policy and optimizer. The policy is Qwen3.5-2B (Qwen Team, 2026), trained with Polar (Xu et al., 2026) and slime/Megatron GRPO (Shoeybi et al., 2019) on four GPUs. We use Dr.GRPO (Liu et al., 2025) (leave-one-out baseline, no standard-deviation normalization), Adam at 10^{-6} with a constant schedule, KL coefficient 0.05, rollout batch size $B=8$, $G=8$ samples per prompt, and 150 optimizer steps. Dynamic sampling drops groups with zero reward standard deviation. Context parallelism is 4 and the maximum response length is 8,192 tokens.

HarnessBandit hyperparameters. Unless noted: $\alpha=0.5$, $\beta=0.3$, $\tau=0.3$, $c=0.2$, $\varepsilon=0.15$, warmup 30 steps, window 32, sketch dimension 65,536, last $K=4$ layers, sketch EMA 0.3.

Comparisons. All trained methods share the optimizer, data product, step budget, and hardware.

- **HarnessBandit:** one harness per batch, selected as in Section 4.
- **Mixed (6 harnesses):** the same 12,000-row file with a global shuffle, so a batch of eight prompts typically mixes several harnesses. No bandit and no gradient sketch.
- **Mixed (OpenClaw+Codex):** the same mixed loader restricted to two harnesses. This tests whether a smaller interface set is sufficient.
- **Qwen3.5-2B (base):** the untuned checkpoint, evaluated with the same serving stack.

5.2 EVALUATION PROTOCOL

Held-out evaluation uses two independent benchmarks. Both use tasks disjoint from ClawGym. They differ in whether the evaluation interface belongs to $\mathcal{H}_{\text{train}}$.

Table 1: Held-out evaluation after ClawGym training. PinchBench: official 147-task OpenClaw suite (held-out tasks, in-distribution harness). ClawEval: `general` split, 161 tasks, native loop (held-out tasks and harness), $N=3$. Pass^3 : all three trials pass. $\text{Pass}@3$: at least one trial passes. Avg score: mean of 161 per-task scores. Same 150-step GRPO budget for trained methods.

Method	PinchBench	ClawEval		
	Mean	Pass^3	$\text{Pass}@3$	Avg score
Qwen3.5-2B (base)	0.330	43/161 (26.7%)	58/161 (36.0%)	0.534
Mixed (OC+Codex)	0.493	32/161 (19.9%)	48/161 (29.8%)	0.490
Mixed (6 harnesses)	0.331	35/161 (21.7%)	62/161 (38.5%)	0.510
HarnessBandit	0.506	43/161 (26.7%)	67/161 (41.6%)	0.556

PinchBench. We report official-suite results on 147 tasks, one run per task, in the official OpenClaw Docker environment (PinchBench, 2026). OpenClaw is one of the six training harnesses, so this suite is out-of-distribution in tasks and in-distribution in interface. The metric is the mean automated task score in $[0, 1]$.

ClawEval. We follow the official protocol on the `general` split (161 tasks) with $N=3$ independent trials (Claw-Eval Team, 2026). Execution uses ClawEval’s own think–act–observe loop and per-task tool schemas, not any member of $\mathcal{H}_{\text{train}}$. The suite is therefore out-of-distribution in both tasks and interface. Pass^3 is the fraction of tasks that pass on all three trials; $\text{pass}@3$ is the fraction that pass on at least one trial; the average score is the mean of the 161 per-task scores.

5.3 RESULTS

Table 1 summarizes the two benchmarks. On PinchBench (held-out tasks, OpenClaw), HarnessBandit records 0.506, a +1.3 point edge over the two-harness OpenClaw–Codex mixture (0.493) and a +17.6 point gap over both the six-harness mixture (0.331) and the untuned base model (0.330). The six-harness mixed-batch run is essentially tied with the base model, so shuffling all six interfaces into mixed batches does not improve the official mean; restricting the mixed loader to OpenClaw and Codex accounts for nearly all of the trained-policy gain on this suite.

On ClawEval (held-out tasks, native loop) the ranking depends on the metric. Pass^3 ties HarnessBandit with the untuned base model at 43/161 (26.7%) and places both mixed schedules below the base model (35 and 32 tasks). $\text{Pass}@3$ and mean score separate the checkpoints: HarnessBandit reaches 67/161 (41.6%) and 0.556, against 62/161 (38.5%) / 0.510 for the six-harness mixture, 58/161 (36.0%) / 0.534 for the base model, and 48/161 (29.8%) / 0.490 for the two-harness mixture. Thus both static mixtures underperform the base model on Pass^3 and on mean score; only HarnessBandit matches the base model on the strict official criterion and exceeds it on $\text{pass}@3$ and average score. PinchBench category means (Appendix B) are highest for HarnessBandit on analysis, log analysis, productivity, research, skills, writing, and integrations. The two-harness mixture is higher on coding, CSV analysis, meeting analysis, and memory. The six-harness mixture wins no category and scores zero on writing.

5.4 BANDIT BEHAVIOR

Figure C.1 (Appendix C) shows the induced schedule. After warmup the mass is non-uniform: OpenClaw, Pi, and OpenCode end near $p = 0.20, 0.20, 0.18$, whereas Qwen Code and Codex end near 0.13, 0.14. Final visit counts range from 21 to 32, a mild reallocation rather than collapse.

Figure 2 shows that the two signals give different rankings. Pi has the highest mean normalized learnability (0.75) but only mid-range transferability (0.49); OpenCode is high on both (0.62, 0.60); OpenClaw’s transferability (0.59) exceeds its learnability (0.55); Qwen Code is lower on transferability (0.37) than learnability (0.53). Thus transferability is not a monotone transformation of learnability in this run. Marker area reflects mean sampling probability p_h , which is largest for OpenCode (0.21) and OpenClaw (0.18).

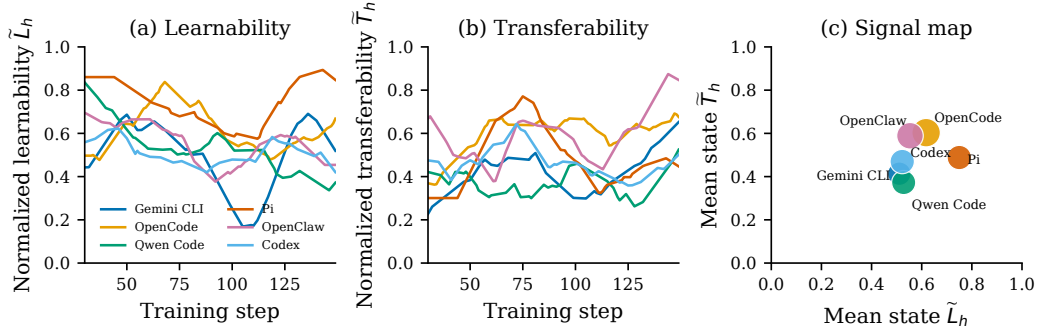


Figure 2: Learnability–transferability analysis for HarnessBandit after warmup. (a,b) 31-step moving averages of the pooled-window normalized signals. (c) Each harness’s mean step-wise scheduler state; marker area is proportional to mean sampling probability.

Appendix E plots directed pairwise gradient cosines. They mostly fluctuate around zero, with both positive and negative intervals and no uniformly beneficial source harness. This fine-grained geometry explains why the scalar transferability state changes over time.

6 CONCLUSION

Multi-harness agentic RL requires a decision that GRPO does not make: which interface should generate the next on-policy batch. We argued that the decision should depend on learnability of the current update and transferability of that update across harnesses. HarnessBandit estimates both from GRPO advantages and sketched last-layer gradients, fuses them after shared-window normalization, and samples a harness-pure batch with a UCB-softmax rule.

After 150 GRPO steps on a ClawGym $\times$ six-harness product, a Qwen3.5-2B policy trained with HarnessBandit records a higher official PinchBench mean than both mixed schedules (held-out tasks, in-distribution OpenClaw). On ClawEval (held-out tasks and harness) it matches the untuned base model on Pass³ (26.7%) while improving pass@3 and mean score, and it exceeds both mixtures on all three ClawEval metrics. The induced schedule is a mild reweighting, not a collapse to one harness.

REFERENCES

- Dimitris Achlioptas. Database-friendly random projections: Johnson–Lindenstrauss with binary coins. *Journal of Computer and System Sciences*, 66(4):671–687, 2003.
- Alon Albalak, Yanai Elazar, Sang Michael Xie, Shayne Longpre, Nathan Lambert, Xinyi Wang, Niklas Muennighoff, Bairu Hou, Aleksandra Piktus, Colin Raffel, et al. A survey on data selection for language models. *Transactions on Machine Learning Research*, 2024.
- Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. In *Machine Learning*, volume 47, pp. 235–256, 2002.
- André Barreto, Will Dabney, Rémi Munos, Jonathan J. Hunt, Tom Schaul, Hado P. van Hasselt, and David Silver. Successor features for transfer in reinforcement learning. *Advances in Neural Information Processing Systems*, 2017.
- Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th International Conference on Machine Learning*, 2009.
- Rich Caruana. Multitask learning. *Machine Learning*, 28:41–75, 1997.
- Moses Charikar, Kevin Chen, and Martin Farach-Colton. Finding frequent items in data streams. *Theoretical Computer Science*, 312(1):3–15, 2004.

-
- Claw-Eval Team. ClawEval: Evaluating personal AI assistants. <https://github.com/claw-eval/claw-eval>, 2026.
- DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Alex Graves, Marc G. Bellemare, Jacob Menick, Rémi Munos, and Koray Kavukcuoglu. Automated curriculum learning for neural networks. *Proceedings of the 34th International Conference on Machine Learning*, 2017.
- William B. Johnson and Joram Lindenstrauss. Extensions of Lipschitz mappings into a Hilbert space. *Contemporary Mathematics*, 26:189–206, 1984.
- Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *Proceedings of the 36th International Conference on Machine Learning*, 2019.
- Tor Lattimore and Csaba Szepesvári. *Bandit Algorithms*. Cambridge University Press, 2020.
- Xiao Liu, Hao Yu, Haohan Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kai Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as agents. *International Conference on Learning Representations*, 2024.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-Zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.
- Sanmit Narvekar, Bei Peng, Matteo Leonetti, Jivko Sinapov, Matthew E. Taylor, and Peter Stone. Curriculum learning for reinforcement learning domains: A framework and survey. In *Journal of Machine Learning Research*, volume 21, pp. 1–50, 2020.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 2022.
- Jiayi Pan, Xingyao Wang, et al. SWE-Gym: An open environment for real-world SWE agents by software engineering Gym. *arXiv preprint arXiv:2412.21139*, 2024.
- Emilio Parisotto, Jimmy Lei Ba, and Ruslan Salakhutdinov. Actor-mimic: Deep multitask and transfer reinforcement learning. In *International Conference on Learning Representations*, 2016.
- PinchBench. PinchBench: A benchmark of real openclaw agent tasks. <https://github.com/pinchbench/skill>, 2026.
- Rémy Portelas, Cédric Colas, Lilian Weng, Katja Hofmann, and Pierre-Yves Oudeyer. Automatic curriculum learning for deep RL: A short survey. *International Joint Conference on Artificial Intelligence*, 2020.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *International Conference on Learning Representations*, 2024.
- Qwen Team. Qwen3.5: Towards native multimodal agents. <https://qwen.ai/blog?id=qwen3.5>, 2026.
- RUC-AIBOX. ClawGym: A benchmark for tool-using agent tasks. <https://huggingface.co/datasets/RUC-AIBOX/ClawGym-Task>, 2025.
- Sebastian Ruder. An overview of multi-task learning in deep neural networks. *arXiv preprint arXiv:1706.05098*, 2017.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

-
- Ozan Sener and Vladlen Koltun. Multi-task learning as multi-objective optimization. *Advances in Neural Information Processing Systems*, 2018.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- Yee Whye Teh, Victor Bapst, Wojciech M. Czarnecki, John Quan, James Kirkpatrick, Raia Hadsell, Nicolas Heess, and Razvan Pascanu. Distral: Robust multitask reinforcement learning. *Advances in Neural Information Processing Systems*, 2017.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. OpenHands: An open platform for AI software developers as generalist agents. *International Conference on Learning Representations*, 2025.
- Sang Michael Xie, Hieu Pham, Xuanyi Xu, Nan Rosemary Liu, Yifeng Du, Niklas Sohoni, Sanni Tu, Kaivalya Narasimhan, Sanni Wang, et al. DoReMi: Optimizing data mixtures speeds up language model pretraining. In *Advances in Neural Information Processing Systems*, 2023.
- Binfeng Xu, Hao Zhang, Shaokun Zhang, Songyang Han, Mingjie Liu, Jian Hu, Shizhe Diao, Zhenghui Jin, Yunheng Zou, Michael Demoret, Jan Kautz, and Yi Dong. Polar: Agentic RL on any harness at scale. *arXiv preprint arXiv:2605.24220*, 2026.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated coding. *Advances in Neural Information Processing Systems*, 2024.
- Yongjin Yang, Jiarui Liu, Yinghui He, Lechen Zhang, Bernhard Schölkopf, and Zhijing Jin. Transferability for general reasoning: An automated curriculum for multi-domain RLVR. *arXiv preprint arXiv:2606.25178*, 2026.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- Shunyu Yao et al. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.
- Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems*, 2014.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning. *Advances in Neural Information Processing Systems*, 2020.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. WebArena: A realistic web environment for building autonomous agents. *International Conference on Learning Representations*, 2024.

A HYPERPARAMETERS

Table A.1 lists the GRPO and scheduler settings shared by every trained method. The optimizer, batch size, sample count, step budget, and hardware are matched so that Table 1 isolates harness allocation. The bandit knobs ($\alpha, \beta, \tau, c, \varepsilon$, warmup, window, sketch) are the values used for the reported HarnessBandit run; mixed-batch baselines ignore them.

Table A.1: Training hyperparameters shared by all GRPO arms.

Item	Value
Base model	Qwen3.5-2B
Algorithm	GRPO / Dr.GRPO (no std. normalization)
Adam learning rate	1×10^{-6} , constant
KL coefficient / type	0.05 / low-var KL
Rollout batch size B	8
Samples per prompt G	8
Optimizer steps	150
Max response / prompt tokens	8,192 / 40,000
Context parallel size	4
GPUs	4
Dynamic sampling	drop zero-std groups; oversample batch 8
$\alpha, \beta, \tau, c, \varepsilon$	0.5, 0.3, 0.3, 0.2, 0.15
Warmup / window	30 / 32
Sketch dim / layers / EMA	65,536 / last 4 / 0.3
Sketch mode	proportional coordinate subsample
Training harnesses	Gemini CLI, OpenCode, Qwen Code, Pi, OpenClaw, Codex

B PINCHBENCH CATEGORY MEANS

Table B.1 breaks the official PinchBench mean into the suite’s task categories. Cell sizes are identical across methods, so differences are not an artifact of a different task mix. HarnessBandit is highest on analysis, integrations, log analysis, productivity, research, skills, and writing. The two-harness OpenClaw–Codex mixture is higher on coding, CSV analysis, meeting analysis, and memory—categories that align with those two interfaces. The six-harness mixture wins no category; its writing cell is exactly zero, which we treat as a recorded score rather than evidence that the mixture cannot write. The overall row reproduces Table 1.

Table B.1: Official PinchBench mean score by category. Cell counts are identical across methods (e.g., coding $n=14$, CSV $n=26$). Overall means match Table 1.

Category	Base	Mixed (OC+Codex)	Mixed (6)	HarnessBandit
analysis	0.368	0.529	0.366	0.579
coding	0.305	0.673	0.558	0.661
csv_analysis	0.148	0.433	0.241	0.352
integrations	0.135	0.191	0.299	0.364
log_analysis	0.397	0.546	0.328	0.568
meeting_analysis	0.389	0.457	0.300	0.444
memory	0.370	0.913	0.750	0.637
productivity	0.336	0.348	0.427	0.485
research	0.418	0.452	0.229	0.477
skills	0.229	0.367	0.560	0.676
writing	0.483	0.581	0.000	0.596
overall	0.330	0.493	0.331	0.506

C BANDIT SCHEDULE

Figure C.1 shows the sampling distribution p_h and the visit counts n_h on the reported run. The first 30 steps are a round-robin warmup, so every harness is observed before softmax takes over. After that line the mass is non-uniform but stays away from a one-arm collapse: OpenClaw, Pi, and OpenCode receive more probability, while Qwen Code and Codex remain above the ε -floor. Visit counts at the end of training span 21–32 (Appendix F), which is a mild reallocation of a 150-step budget.

D RAW ONLINE SIGNALS

Figure D.1 plots the un-normalized observations that later become $\tilde{L}_h$ and $\tilde{T}_h$. Learnability L_h is a mean absolute advantage and stays clearly positive; transferability T_h is an average cosine and

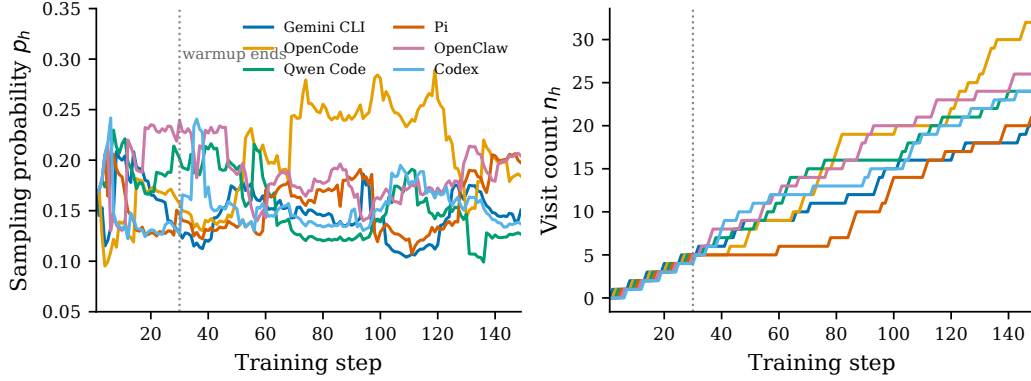


Figure C.1: HarnessBandit on the reported run. Left: sampling probabilities. Right: visit counts. The vertical line marks the end of the 30-step round-robin warmup.

fluctuates in a narrow band around zero. The two traces therefore live in different units, which is why Section 4.4 applies a shared sliding-window min–max before fusion. Rankings also differ: a harness can look learnable on the left panel while sitting near the bottom of the right panel. Figure 2 in the main text shows the same run after normalization.

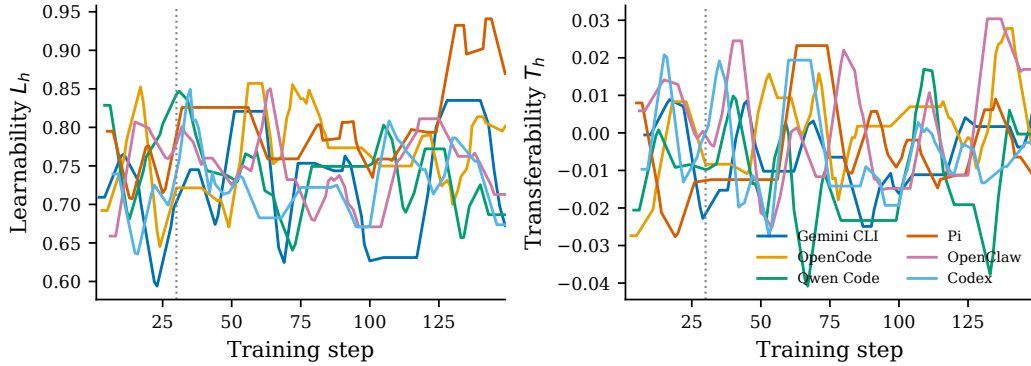


Figure D.1: Online signals on the HarnessBandit run (7-step moving average). Left: learnability L_h . Right: transferability T_h .

E PAIRWISE GRADIENT GEOMETRY

The scalar T_{h_t} averages the current sketch against every other harness EMA. Figure E.1 unrolls that average into directed pairs. Each panel fixes the source harness; each line is the cosine from that source’s current sketch to one peer EMA. The traces move through both positive and negative intervals and do not identify a source that is aligned with all peers at all times. That geometry is why the fused transferability state in Figure 2 evolves rather than locking onto a single “helpful” interface.

F FINAL HARNESSBANDIT STATE

These are the terminal p_h and n_h of Figure C.1, recorded after the last optimizer step. Sampling probabilities are OpenClaw 0.204, Pi 0.197, OpenCode 0.184, Gemini CLI 0.151, Codex 0.137, Qwen Code 0.127. Visit counts are OpenCode 32, OpenClaw 26, Qwen Code 24, Codex 24, Gemini CLI 21, Pi 21 (148 observations after warmup). Uniform allocation over six arms would give

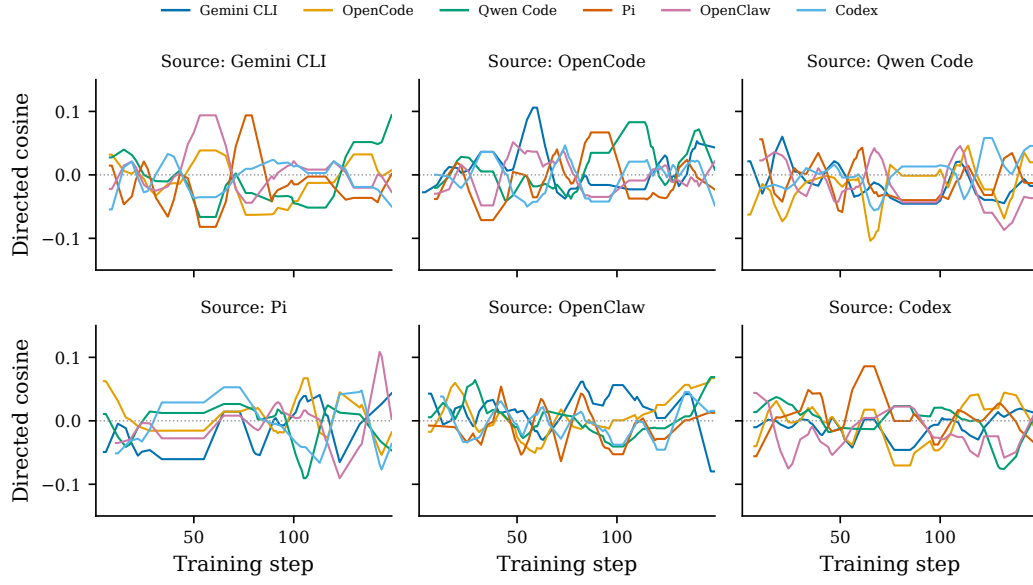


Figure E.1: Pairwise analysis of directed current-to-peer gradient cosines (10-step moving average). Each panel fixes the source harness; line color denotes the peer EMA.

$p_h = 1/6 \approx 0.167$ and about 25 visits each; the observed schedule is a tilt toward OpenClaw, Pi, and OpenCode, not a collapse to one arm. Pi's visit count is low relative to its final p_h because probability mass arrived late.